

Global energy: AWS lakehouse platform

01

Lead Data Engineer (contract) London, UK, Sep 2024 to Oct 2025

Project scope
Smart metering
Generation assets
Energy trading
Customer analytics
London, UK

Energy platform modernisation
Meter readings, generation telemetry, trading and consumption data.
AWS batch lakehouse and operational streaming.
Consumers: commercial analytics, operations and regulatory reporting.

Team and responsibilities

Six-member data engineering team (CV).
Own production release governance, code reviews, deployment approvals, validation and rollback readiness.
Mentor engineers in Python, PySpark, SQL optimisation, data modelling and CI/CD.

Agile delivery and releases

Suggested interview walkthrough

1. Refinement and planning
Agree source contracts, acceptance criteria and reconciliation rules.
2. Build and review
Develop ingestion, Glue jobs and MWAA DAGs.
Review code and data models.
3. Validation and release
Reconcile outputs. Deploy through GitHub Actions with approvals and rollback readiness.
4. Retrospective
Review incidents and improve quality / performance.

CV confirms dates and team. Schemas illustrate a design. Agile cadence, deployment details and SLAs remain to confirm.

Batch ingestion: reference and master data

02

Illustrative schemas: F = validated full refresh, I = watermark / file increment, C = optional CDC. Initial snapshot establishes a baseline.

METER [F/I]

PK meter_id
FK supply_point_id
serial_no, meter_type, status
installed_at, source_updated_at

CUSTOMER [I/C]

PK customer_id
customer_type, region
PII stored under restricted access
valid_from, valid_to, updated_at

ACCOUNT [I/C]

PK account_id; FK customer_id
account_ref, status
opened_at, closed_at
source_updated_at

SUPPLY_POINT [F/I]

PK supply_point_id
supply_ref, region, grid_zone
supply_type, status
source_updated_at

METER_ASSIGNMENT [I/C]

PK assignment_id; FK meter_id
FK account_id, supply_point_id
valid_from, valid_to
source_updated_at

TARIFF [F/I]

PK tariff_id
name, tariff_type, currency
valid_from, valid_to
source_updated_at

TARIFF_RATE [F/I]

PK rate_id; FK tariff_id
unit_rate, standing_charge
unit, time_band
valid_from, valid_to

GENERATION_ASSET [F/I]

PK asset_id; FK site_id
asset_type, rated_capacity_kw
commissioned_at, status
source_updated_at

SITE [F]

PK site_id
site_name, region
location_ref, time_zone
source_updated_at

SETTLEMENT_CALENDAR [F]

PK (settlement_date, period)
start_utc, end_utc
local_start, duration_minutes
DST_flag, calendar_version

PRODUCT_MARKET [F]

PK product_market_id
product_code, market_code
delivery_unit, price_currency
effective_from, effective_to

REFERENCE_CODES [F]

PK (code_type, code)
description, valid_from, valid_to
Unit, reading quality, event type
Report line / rule definitions

Source DB / API / files → Databricks Python / PySpark → S3 Bronze → AWS Glue / Iceberg → Silver / Gold

Full refresh small reference tables into a staged snapshot, validate then publish. CUSTOMER, ACCOUNT and METER assignments need history. Use I or C when change tracking exists.

Audit fields: source_system, source_record_id, source_updated_at, ingested_at, batch_run_id, operation. Retain immutable raw extracts. Register managed Iceberg tables in Glue Data Catalog.

Batch ingestion: transactions and telemetry

03

Illustrative schemas: I = incremental extraction, A = immutable append, C = optional CDC for mutable records. Actual source DDL needs confirmation.

METER_READING [I/A]

PK reading_id; FK meter_id
reading_ts, interval_end, version
consumption_kwh, reading_type
quality_code, source_updated_at

READING_CORRECTION [I/A]

PK correction_id; FK reading_id
meter_id, reading_ts, version
replacement_kwh, reason
corrected_at, source_updated_at

GRID_EVENT [A]

PK event_id; FK supply_point_id
meter_id, event_ts, event_type
severity, event_payload_ref
source_offset, schema_version

DEVICE_STATUS [I/A]

PK (meter_id, observed_at)
FK meter_id
signal_source, battery_pct
health_status, source_offset

GENERATION_TELEMETRY [I/A]

PK (asset_id, observed_at)
FK asset_id
output_kwh, available_minutes
quality_code, source_updated_at

ASSET_AVAILABILITY [I/C]

PK availability_id; FK asset_id
start_at, end_at, status
capacity_available_kw, reason
source_updated_at

ENERGY_TRADE [I/C]

PK trade_id
FK product_market_id
executed_at, counterparty_id
quantity_mwh, price, status

TRADE_DELIVERY_LEG [I/C]

PK leg_id; FK trade_id
delivery_start, delivery_end
quantity_mwh, price, currency
source_updated_at

BILL [I/C]

PK bill_id; FK account_id
issue_date, service_period
net_amount, tax, currency
status, source_updated_at

BILL_LINE [I/C]

PK line_id; FK bill_id, tariff_id
meter_id, period_start, period_end
billed_kwh, net_amount, tax
charge_type, source_updated_at

CONSUMPTION_RECORD [I]

PK consumption_id; FK meter_id
account_id, settlement_date
interval_kwh, estimate_flag
version, source_updated_at

RECONCILIATION_CONTROL [I/A]

PK (source, period, control, run)
expected_count, expected_total
unit, source_cutoff
created_at, control_version

Source DB / API / files → Databricks Python / PySpark → S3 Bronze → AWS Glue / Iceberg → Silver / Gold

Watermark on source_updated_at with overlap and deterministic deduplication. File feeds use manifests / checksums. CDC captures deletes where available. Commit controls only after successful landing.

MWAA schedules Databricks ingestion and Glue jobs. Proposed controls: ingestion_config, watermark_control, batch_audit, schema_contract, reconciliation_result. DynamoDB controls are quite details.

Streaming ingestion: smart meters and grid events

04

Producers and event contracts

Meter head-end publisher

Smart meter readings and corrections.

Related sources: METER_READING, READING_CORRECTION, METER, METER_ASSIGNMENT.

Grid / operational event publisher

Voltage dip, suspected outage and device status events.

Related sources: GRID_EVENT, DEVICE_STATUS. Confirm which events the publisher exposes.

Generation telemetry

Batch ingestion is confirmed in the CV. Streaming from SCADA / gateway is an optional extension, requiring a producer and source contract.

CDC is a database change feed. Meter telemetry arrives from event publishers. Use a CDC connector only for mutable database tables when available.

Event fields: event_id, meter_id, reading_ts, consumption_kwh, reading_type, signal_source, grid_event_flag, schema_version, source_offset. Partition by meter_id when per-meter ordering matters.

Kafka and Delta streaming are CV-confirmed. Broker count, throughput, trigger interval, watermark duration, DynamoDB and SLA targets require confirmation.

Transport, processing and destinations

CV-CONFIRMED STREAM PATH

Head-end / grid publisher → Apache Kafka → Databricks PySpark Structured Streaming → Delta Lake on S3 → operational dashboards.

Process event time, validate contracts, enrich from meter references, deduplicate and checkpoint to S3. Quarantine rejected events with reasons.

GUIDE DESIGN DETAILS TO CONFIRM

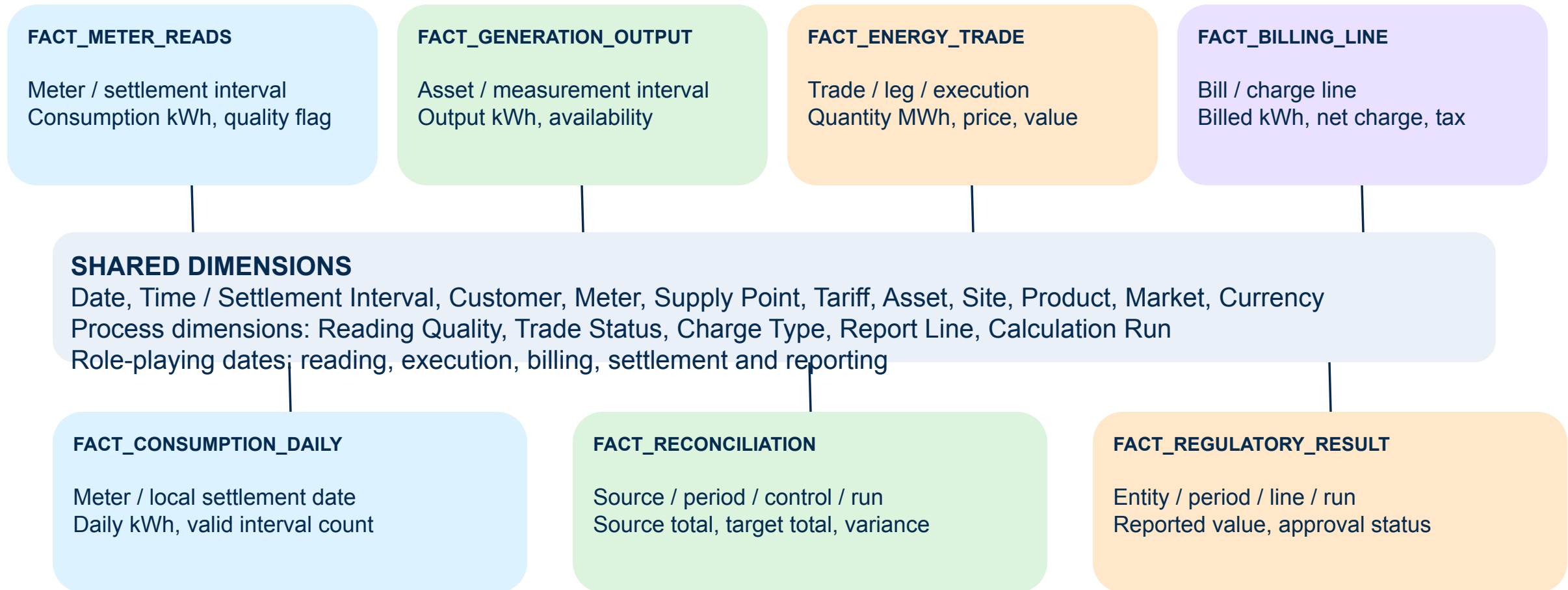
Avro / Schema Registry, raw / enriched / DLQ topics. DynamoDB provides an optional hot lookup path with idempotent, version-aware writes.

Batch integration: read Delta through a supported reader, reconcile overlaps with batch source keys, then write curated Iceberg. Delta and Iceberg use distinct table locations.

Consumer data model: energy fact constellation

05

Seven proposed facts share conformed dimensions. Metering and generation are guide models. Billing, trading and reporting contracts extend the model for the CV scope.



Illustrative consumer schema. Different grains require separate facts. Generation, trading and metering measures need agreed units and reconciliation rules.

Consumer facts: keys, grains and measures

06

FACT_METER_READS

Grain: meter × settlement interval
PK reading_sk, unique grain key
FK meter, customer, tariff, interval, quality
Measures: consumption_kwh, read_count
Attributes: reading_type, version
Resolve corrections before publishing

FACT_GENERATION_OUTPUT

Grain: asset × measurement interval
PK generation_sk, unique grain key
FK asset, site, interval, unit, quality
Measures: output_kwh, available_minutes
Keep capacity in the asset dimension
Utilisation uses an agreed denominator

FACT_ENERGY_TRADE

Grain: trade × leg × execution
PK trade_sk, source trade / leg key
FK product, market, execution / delivery
FK counterparty, currency, status
Measures: quantity_mwh, price, value
Price is non-additive. Preserve revisions

FACT_BILLING_LINE

Grain: bill × charge line
PK billing_line_sk, source line key
FK customer, account, tariff, charge type
FK issue / service dates, currency
Measures: billed_kwh, net_amount, tax
Separate reversal / credit adjustment

FACT_CONSUMPTION_DAILY

Grain: meter × settlement date
PK daily_consumption_sk
FK meter, customer, tariff, date, quality
Measures: daily_kwh, valid_intervals
Derived from accepted interval reads
Do not union daily and interval facts

FACT_RECONCILIATION

Grain: source × period × control × run
PK reconciliation_sk
FK source, period, rule, calculation run
Measures: source_total, target_total
Variance_amount, variance_pct, counts
Retain status, exceptions and evidence

SCD Type 2 retains Customer / Meter assignment / Tariff history. Resolve event-effective surrogate keys during loading. Store UTC event time plus the settlement calendar. DST can change daily interval counts.

Reporting model: reconciliation and audit evidence

07

FACT_REGULATORY_RESULT

Grain: reporting entity × period × report line × unit / currency × calculation run

PK regulatory_result_sk

FK entity_sk, period_sk, report_line_sk, unit_sk, currency_sk, calculation_run_sk

Measures: reported_value, adjustment_value

Context: definition version, source cutoff, approval status and publication timestamp

Dimension and relationship rules

Dimension PK to fact FK uses 1:N relationships.

Keep customer PII in restricted dimensions. Use business keys to match sources and surrogate keys for analytical joins.

Customer / Tariff / Meter assignments retain effective dates.

Facts use the correct historical version.

Aggregate each fact to an agreed reporting grain before combining results. Avoid direct fact joins that multiply energy or money.

Coverage: metering, generation, billing / consumption, trading and report preparation. The model supports Ofgem-related analytics. Submission definitions and compliance approval remain business-owned.

Reporting and reconciliation controls

Compare meter / consumption totals with billing controls at a compatible scope. Account for estimated reads, corrections, units and cutoffs.

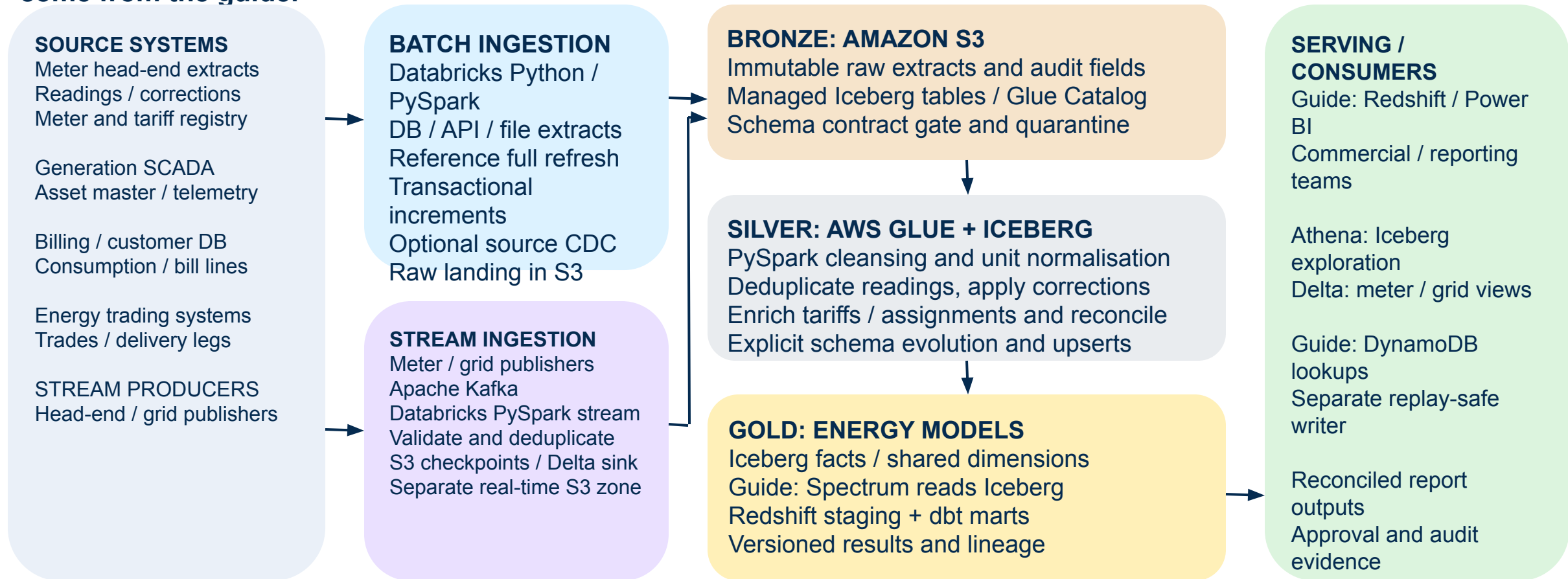
BRIDGE_REPORT_CONTRIBUTION links a result to contributing records, rules and allocation weights when needed.

Retain accepted / rejected counts, exception details, source totals, report versions and approval evidence. Business owners define thresholds.

Global energy: AWS lakehouse architecture

08

CV-confirmed core: Databricks ingestion, S3, Glue, Iceberg, Kafka / Delta, MWA and Terraform. Serving details come from the guide.



Batch: source extracts → Databricks → S3 Bronze → Glue / Iceberg Silver / Gold → warehouse / reporting

Streaming: publishers → Kafka → Structured Streaming → S3 Delta → dashboards. Reconcile into Iceberg through batch.

MWA schedules dependencies / retries. Terraform and GitHub Actions manage releases. IAM, Secrets Manager and network controls protect access. Guide extensions: KMS, Lake Formation and monitoring.

1. Source contracts and batch

Databricks ingestion lands meter, generation, trading and consumption data in S3. Full-refresh small references after validation. Watermark transactions with overlap. Preserve keys, source timestamps and raw extracts. MWAA coordinates dependencies.

3. Bronze and Silver processing

S3 Bronze supports replay. Glue jobs cleanse and enrich Iceberg tables, normalise units and retain corrections. Schema gates accept approved compatible changes. Breaking changes enter quarantine. Record rejection reasons and source-to-target reconciliation.

5. Reporting and data quality

Validate uniqueness, mandatory values, relationships, freshness and totals. Compare consumption with billing at the same scope and cutoff. Approved report definitions drive aggregation. Retain contribution lineage, exception resolution and publication approvals.

2. Streaming and table formats

Kafka transports head-end / grid events. Structured Streaming validates and deduplicates them, with checkpoints and Delta on S3. Keep Delta and Iceberg table locations separate. Reconcile events and batch extracts by meter / interval / version before curated upserts.

4. Gold and warehouse serving

Facts retain explicit grains and event-effective dimensions. Guide serving uses Spectrum to read Iceberg, staging / materialising data into Redshift before dbt marts. Athena supports exploration. Snapshot references and report runs preserve reproducibility.

6. Reliability and releases

MWAA owns schedules, sensors, retries and SLA alerts. Terraform provisions AWS resources. GitHub Actions supports reviewed releases and rollback readiness. Use scoped IAM and Secrets Manager. DynamoDB is an optional guide hot path with replay-safe writes.